ODOO DEVELOPER PROGRAM

Environment & Decorators

Odoo 17 Development Tutorials

Ahmed Muhumed

July 26, 2026

Outline

1. Introduction to the Environment
2. **`self.env`** Explained
3. Environment Components
4. Accessing Models with `env[...]`
5. User and Company
6. Context (`env.context`)
7. `with_*` Environment Methods
8. **`sudo()`** Superuser Access
9. `env.ref()` and XML IDs
10. Introduction to API Decorators
11. `@api.model` and `@api.model_create_multi`
12. `@api.depends`
13. `@api.onchange`
14. `@api.constrains`
15. Other Important Decorators
16. Summary

Introduction to the Environment

What Will We Learn?

- What **`self.env`** is and why every Odoo method uses it.
- The main parts of the environment: user, company, context, cursor, and registry.
- How to switch environment safely with `with_context`, `with_user`, `sudo()`, and friends.
- How to resolve XML IDs with `env.ref()`.
- The most important **`@api`** decorators and when to use each one.

Why the Environment Matters

- In Odoo, business code almost never talks to PostgreSQL directly.
- Every recordset carries an **environment**.
- The environment answers questions such as:
  - Who is the current user?
  - Which company is active?
  - What language / context flags are active?
  - Which database cursor should be used?
- Understanding `self.env` is the foundation for correct ORM code.

`self.env` Explained

What Is self.env?

- **self.env** is the **Environment** object attached to a recordset.
- It is the gateway to models, users, companies, context, and the database cursor.

Basic Access

```
class Student(models.Model):
    _name = 'student.student'

    def action_demo(self):
        print(self.env)
        students = self.env['student.student'].search([])
```

- Inside model methods, you almost always start from `self.env`.

Environment vs Recordset

- A **recordset** represents one or more records of a model.
- An **environment** is the execution context around those records.

```
student = self.env['student.student'].browse(15)
print(student)      # student.student(15,)
print(student.env)  # Environment object
```

- Different recordsets may share the same environment.
- Changing the environment (for example with `sudo()`) returns a **new** recordset / env pair.

Where Does `self.env` Come From?

- Odoo builds the environment when a request / shell / cron job starts.
- Then every recordset created in that flow receives that environment.

```
# In Odoo shell
env['student.student'].search([])

# In a model method
self.env['student.student'].search([])
```

- In both cases, `env` is the same kind of object.
- In model methods, it is available as **`self.env`**.

Environment Components

Main Parts of the Environment

- The environment contains several important pieces:
  1. **`env.cr`** – database cursor
  2. **`env.uid`** – current user ID
  3. **`env.user`** – current user record
  4. **`env.context`** – context dictionary
  5. **`env.company`** / **`env.companies`**
  6. **`env.registry`** – model registry
  7. **`env.su`** – whether sudo mode is active

Breaking Down env.cr

- **env.cr** is the PostgreSQL cursor for the current transaction.

```
self.env.cr.execute("SELECT COUNT(*) FROM student_student")
count = self.env.cr.fetchone()[0]
```

- Prefer ORM methods over raw SQL whenever possible.
- Use `env.cr` only for advanced queries/reports.
- Odoo manages commit/rollback for HTTP requests automatically.

Breaking Down `env.uid` and `env.su`

- **`env.uid`**: integer ID of the current user.
- **`env.su`**: `True` if the environment is in superuser mode.

```
print(self.env.uid)   # e.g. 2
print(self.env.su)    # False in normal requests
```

- After `sudo()`, `env.su` becomes `True`.
- Access rights and record rules depend on these values.

Breaking Down env.registry

- **env.registry** holds all loaded models for the database.

```
Student = self.env.registry['student.student']
```

- In normal business code, prefer:

```
Student = self.env['student.student']
```

- The registry is rebuilt when modules are installed or updated.

Accessing Models with `env[...]`

The Pattern `self.env['model.name']`

- This is the standard way to get a model from the environment.

Method Syntax

```
Model = self.env['student.student']
records = Model.search([])
```

- `self.env['student.student']` returns an empty recordset of that model in the current environment.
- From there you call ORM methods: `create`, `search`, `browse`, ...

Practical Examples

```python
# Search students
students = self.env['student.student'].search([('active', '=', True)])

# Create a classroom
classroom = self.env['school.classroom'].create({
    'name': 'Grade 12-A',
})

# Browse by ID
partner = self.env['res.partner'].browse(7)
```

- Always use the technical model name (`_name`), not the Python class name.

Same Model, Different Environments

```python
students = self.env['student.student']
admin_students = self.env['student.student'].sudo()
company_students = self.env['student.student'].with_company(company)
```

- All three access the same model.
- But rights, company, or context may differ.
- This is why environment control is so important.

User and Company

- **env.user** is the current `res.users` record.

```
user = self.env.user
print(user.name)
print(user.login)
print(user.has_group('base.group_system'))
```

- Useful for:
  - checking groups / permissions in Python
  - writing the current user into a field
  - personalizing business logic

- **env.company**: the active company.
- **env.companies**: allowed companies in the current environment.

```
company = self.env.company
print(company.name)
print(self.env.companies.mapped('name'))
```

- Multi-company rules often depend on these values.
- Never hard-code company IDs when `env.company` is enough.

Practical User / Company Patterns

```python
def create(self, vals):
    vals.setdefault('company_id', self.env.company.id)
    vals.setdefault('user_id', self.env.user.id)
    return super().create(vals)
```

- Set defaults from the environment.
- Keep business documents linked to the correct company/user.

Context (`env.context`)

What Is the Context?

- **`env.context`** is a dictionary of flags and values for the current operation.

```
print(self.env.context)
# example: {'lang': 'en_US', 'tz': 'Africa/Mogadishu', 'uid': 2}
```

- Context can change:
  - language
  - default values
  - active test behavior
  - custom business flags

Reading Context Values

```
lang = self.env.context.get('lang')
force_email = self.env.context.get('force_send_email')
default_gender = self.env.context.get('default_gender')
```

- Use `.get()` so missing keys do not crash your code.
- Keys starting with `default_` are used by `default_get()`.

Method Syntax

```
records.with_context(key=value, ...)
# or
records.with_context({...})
```

```
students = self.env['student.student'].with_context(
    default_gender='female',
    active_test=False,
)
```

- Returns a new recordset/environment with updated context.
- Original environment is not modified.

Common Context Keys

- `lang` – language code
- `tz` – timezone
- `active_test` – whether to hide inactive records
- `default_fieldname` – default value for create forms
- custom keys for your own business logic

```
self.with_context(skip_student_validation=True).write(vals)
```

`with_*` Environment Methods

- Runs the next operations as another user.

```
other_user = self.env['res.users'].browse(8)
students = self.env['student.student'].with_user(other_user).search([])
```

- Useful for testing access rights or impersonation flows.
- Access rights and record rules follow the selected user.

WITH_COMPANY() AND WITH_COMPANIES()

```
company = self.env['res.company'].browse(2)
Student = self.env['student.student'].with_company(company)
Student.create({'name': 'Ahmed', 'company_id': company.id})
```

- Changes the active company in the environment.
- Important in multi-company databases.
- Prefer this over manually guessing company context.

WITH_ENV() AND CHAINING

```
new_env = self.env(user=other_user, context=dict(self.env.context, lang='so_SO'))
students = self.env['student.student'].with_env(new_env).search([])
```

- `with_env()` replaces the whole environment.
- You can also chain helpers:

```
self.env['student.student'] \
    .with_user(user) \
    .with_company(company) \
    .with_context(lang='en_US') \
    .search([])
```

Important Rule

- Environment helpers return **new** recordsets.
- They do not mutate the old one in place.

```python
a = self.env['student.student']
b = a.sudo()
print(a.env.su)  # False
print(b.env.su)  # True
```

- Always keep using the returned value.

`sudo()` Superuser Access

What Is sudo()?

- **sudo()** returns the same records in a superuser environment.
- Superuser bypasses access rights and record rules.

Method Syntax

```
records.sudo()
records.sudo(flag=True)
records.sudo(False)  # leave sudo mode
```

When to Use sudo()

- System operations that normal users must trigger but cannot fully access.
- Creating technical records (followers, logs, queues) in controlled code.
- Cron / server actions that need broader access.

```
# Controlled technical write
self.sudo().write({'system_note': 'Processed by cron'})
```

When NOT to Use `sudo()`

- Do not use `sudo()` just to “make the error go away”.
- Do not expose sudo results directly to untrusted UI input.
- Prefer correct access rights and record rules when possible.

```
# Dangerous pattern
self.env['res.partner'].sudo().search([]).unlink()
```

- Ask: does this really need superuser power?

SUDO() VS WITH_USER()

- `sudo()`: bypass security (superuser).
- `with_user(user)`: act as that user, still under that user's rights.

```
# Bypass rights
self.sudo().write({'state': 'done'})

# Act as teacher user, respecting rights
self.with_user(teacher).write({'state': 'done'})
```

`env.ref()` and XML IDs

What Is env.ref()?

- **env.ref()** finds a record by its XML / external ID.

Method Syntax

```
record = self.env.ref('module_name.xml_id')
record = self.env.ref('module_name.xml_id', raise_if_not_found=False)
```

- Extremely common for groups, stages, sequences, default records, and actions.

Practical Examples

```
admin_group = self.env.ref('base.group_system')
student_action = self.env.ref('school_manager.action_student')
draft_stage = self.env.ref('project.project_stage_0', raise_if_not_found=False)

if self.env.user.has_group('base.group_system'):
    # admin-only logic
    pass
```

Why Prefer XML IDs over Hard-Coded IDs?

- Database IDs change between databases.
- XML IDs are stable across installs when defined in data files.

```
# Bad
stage = self.env['project.task.type'].browse(3)

# Good
stage = self.env.ref('project.project_stage_0')
```

- Use `raise_if_not_found=False` when the record may be optional.

Other Useful Environment Helpers

```
self.env.lang            # current language, e.g. 'en_US'
self.env.is_superuser()
self.env.is_admin()
self.env.is_system()
```

- These helpers make environment checks clearer and safer.

Introduction to API Decorators

What Are API Decorators?

- In Odoo, methods are often decorated with `@api....` markers.
- A decorator tells Odoo **how** the method should be called and when it should run.
- Decorators are not just Python sugar – they change ORM behavior.

Common Decorators We Will Cover

- `@api.model`
- `@api.model_create_multi`
- `@api.depends`
- `@api.onchange`
- `@api.constrains`
- `@api.ondelete`
- `@api.autovacuum`
- `@api.private` / `@api.readonly`

Why Decorators Matter

- They decide whether a method is model-level or record-level.
- They connect computed fields to their dependencies.
- They validate data before it is saved.
- They control UI onchange behavior in forms.
- Using the wrong decorator creates subtle bugs.

`@api.model` and `@api.model_create_multi`

- Marks a method as a **model-level** method.
- `self` is an empty recordset of the model, not one specific record.

Method Syntax

```
@api.model
def my_method(self, values):
    return self.create(values)
```

- Common for helpers that create/search without needing an existing record.

Breaking Down @api.model

```python
@api.model
def create_demo_student(self):
    print(self)  # student.student()
    return self.create({'name': 'Demo Student'})
```

- Call it from the model:

```python
self.env['student.student'].create_demo_student()
```

- Do not expect `self.id` to exist inside a pure model method.

- Used for efficient multi-record creation.
- The argument is a **list of dictionaries**.

```python
@api.model_create_multi
def create(self, vals_list):
    for vals in vals_list:
        vals.setdefault('active', True)
    return super().create(vals_list)
```

- Preferred modern override for `create()` in Odoo 17.

@api.model vs Record Methods

- Model method:

```python
@api.model
def get_active_count(self):
    return self.search_count([('active', '=', True)])
```

- Record method:

```python
def action_archive(self):
    self.ensure_one()
    self.active = False
```

- Choose based on whether you need existing records.

@api.depends

What Is @api.depends?

- Used on **computed fields**.
- Tells Odoo which fields trigger recomputation.

Method Syntax

```
age_group = fields.Char(compute='_compute_age_group')

@api.depends('age')
def _compute_age_group(self):
    for student in self:
        student.age_group = 'adult' if student.age >= 18 else 'minor'
```

Breaking Down Dependencies

```python
@api.depends('birth_date')
def _compute_age(self):
    ...

@api.depends('line_ids.price', 'line_ids.quantity')
def _compute_total(self):
    ...

@api.depends('classroom_id.teacher_id.name')
def _compute_teacher_name(self):
    ...
```

- You can depend on related paths and one2many subfields.
- Missing dependencies cause stale computed values.

Stored vs Non-Stored Computed Fields

```python
# Recomputed on the fly
display_label = fields.Char(compute='_compute_display_label')

# Stored in database and recomputed when deps change
total = fields.Float(compute='_compute_total', store=True)
```

- `store=True` needs accurate `@api.depends`.
- Non-stored fields are calculated when read.

Inverse and Related Notes

```python
@api.depends('first_name', 'last_name')
def _compute_name(self):
    for rec in self:
        rec.name = f"{rec.first_name} {rec.last_name}"

def _inverse_name(self):
    for rec in self:
        parts = (rec.name or '').split(' ', 1)
        rec.first_name = parts[0]
        rec.last_name = parts[1] if len(parts) > 1 else ''
```

- Inverse methods allow writing back into computed fields.

`@api.onchange`

What Is @api.onchange?

- Runs in the **UI form** when a field value changes.
- Used to update other fields before the record is saved.

Method Syntax

```
@api.onchange('classroom_id')
def _onchange_classroom_id(self):
    if self.classroom_id:
        self.teacher_id = self.classroom_id.teacher_id
```

Breaking Down Onchange Behavior

- Onchange methods work on a temporary record in the form client.
- They do **not** automatically run during pure Python `create()/write()` calls.
- Return warnings or domain updates when needed:

```python
@api.onchange('age')
def _onchange_age(self):
    if self.age and self.age < 0:
        return {
            'warning': {
                'title': 'Invalid Age',
                'message': 'Age cannot be negative.',
            }
        }
```

Returning Domain from Onchange

```python
@api.onchange('school_id')
def _onchange_school_id(self):
    self.classroom_id = False
    return {
        'domain': {
            'classroom_id': [('school_id', '=', self.school_id.id)]
        }
    }
```

- Useful for dynamic dropdown filtering in forms.

@API.ONCHANGE VS @API.DEPENDS

- `@api.onchange`: UI helper before save.
- `@api.depends`: computation/recompute logic in ORM.

```python
# UI convenience
@api.onchange('classroom_id')
def _onchange_classroom_id(self):
    self.teacher_id = self.classroom_id.teacher_id

# Real stored/computed business value
@api.depends('classroom_id.teacher_id')
def _compute_teacher_id(self):
    for rec in self:
        rec.teacher_id = rec.classroom_id.teacher_id
```

@api.constrains

What Is @api.constrains?

- Used for **Python validation** when fields change.
- Runs during `create()` / `write()` when watched fields are involved.

Method Syntax

```
@api.constrains('age')
def _check_age(self):
  for student in self:
    if student.age < 0:
      raise ValidationError("Age cannot be negative.")
```

Breaking Down Constraint Methods

```python
from odoo.exceptions import ValidationError

@api.constrains('email')
def _check_email(self):
  for student in self:
    if student.email and '@' not in student.email:
      raise ValidationError("Please provide a valid email.")
```

- Always loop over `self`.
- Raise `ValidationError` for business validation failures.
- Constraints must be deterministic and side-effect free.

Multi-Field Constraints

```python
@api.constrains('start_date', 'end_date')
def _check_dates(self):
  for rec in self:
    if rec.start_date and rec.end_date and rec.end_date < rec.start_date:
      raise ValidationError("End date must be after start date.")
```

- List every field that should trigger the check.
- If a field is missing from `@api.constrains`, changes to it may skip validation.

Python Constraints vs SQL Constraints

- `@api.constrains`: flexible Python checks, rich error messages.
- `_sql_constraints`: database-level checks/uniqueness, very strong.

```
_sql_constraints = [
  ('email_unique', 'unique(email)', 'Email must be unique.'),
]
```

- Use both when needed.

Other Important Decorators

- Controls deletion restrictions in modern Odoo versions.

```python
@api.ondelete(at_uninstall=False)
def _unlink_except_done(self):
    for rec in self:
        if rec.state == 'done':
            raise UserError("Done records cannot be deleted.")
```

- Clearer than putting all logic only in `unlink()`.

- Marks a method for periodic cleanup by the autovacuum cron.

```python
@api.autovacuum
def _gc_old_logs(self):
    old = self.search([
        ('create_date', '<', fields.Datetime.subtract(fields.Datetime.now(), days=90)),
    ])
    old.unlink()
```

- Useful for temporary logs, transient leftovers, and cleanup tasks.

@API.PRIVATE AND @API.READONLY

```python
@api.private
def _prepare_student_vals(self, values):
    values = dict(values)
    values.setdefault('active', True)
    return values

@api.readonly
def get_report_data(self):
    return self.read(['name', 'age'])
```

- `@api.private`: not available through RPC / external API.
- `@api.readonly`: declares a non-writing method (useful for routing/optimization).

@API.RETURNS

```python
@api.returns('self', lambda value: value.id)
def copy(self, default=None):
    return super().copy(default)
```

- Documents / adapts return values for RPC compatibility.
- Seen on methods that return records but need IDs over XML-RPC.

Summary

Environment Summary

- **`self.env`** is the gateway to models, users, companies, context, and the cursor.
- Use `env['model']`, `env.user`, `env.company`, `env.context`, and `env.ref()`.
- Switch environment with `with_context`, `with_user`, `with_company`, and `sudo()`.
- Environment helpers return new recordsets; keep the returned value.

Decorator Summary

- `@api.model` / `@api.model_create_multi` – model-level methods and create overrides.
- `@api.depends` – computed field dependencies.
- `@api.onchange` – UI form updates before save.
- `@api.constrains` – Python validation on create/write.
- `@api.ondelete`, `@api.private`, `@api.readonly`, `@api.autovacuum` – specialized behaviors.

Practical Checklist

- Need another model? → `self.env['model.name']`
- Need current user/company? → `env.user` / `env.company`
- Need temporary flags? → `with_context(...)`
- Need to bypass rights carefully? → `sudo()`
- Need stable record lookup? → `env.ref('module.xml_id')`
- Need compute / onchange / validation? → choose the correct decorator